МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение
высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»
Институт перспективной инженерии
Межинститутская базовая кафедра
Институт информационных технологий и кибербезопасности

Отчёт к лабораторной работе №1

Тема: **«Тестирование чёрного и белого ящика»**

Дисциплина: **«Тестирование и отладка программного обеспечения»**

Выполнил:

Студент группы ПИЖ-б-о-23-1
направление подготовки:
09.03.04 «Программная
инженерия»

Астраков Борис Александрович

_____

Проверил:

Ассистент МИБК

Потапов Иван Романович

_____

Ставрополь 2026

Цель работы: для разработанного приложения продемонстрировать на выбор применение технологии тестирования «черного ящика» или «белого ящика».

Ход работы:

Вариант проекта – игра «Собиратель монет» на pygame.

1. Тестирование «чёрного ящика»

Чёрный ящик: тестировщик оценивает поведение приложения по входам и выходам, не изучая исходный код. Цель — проверить соответствие требованиям и корректность функциональности.

Сценарий тесткейса – сбор всех монет на уровне.

Последовательность тестирования:

1) Запустить игру и начать уровень;

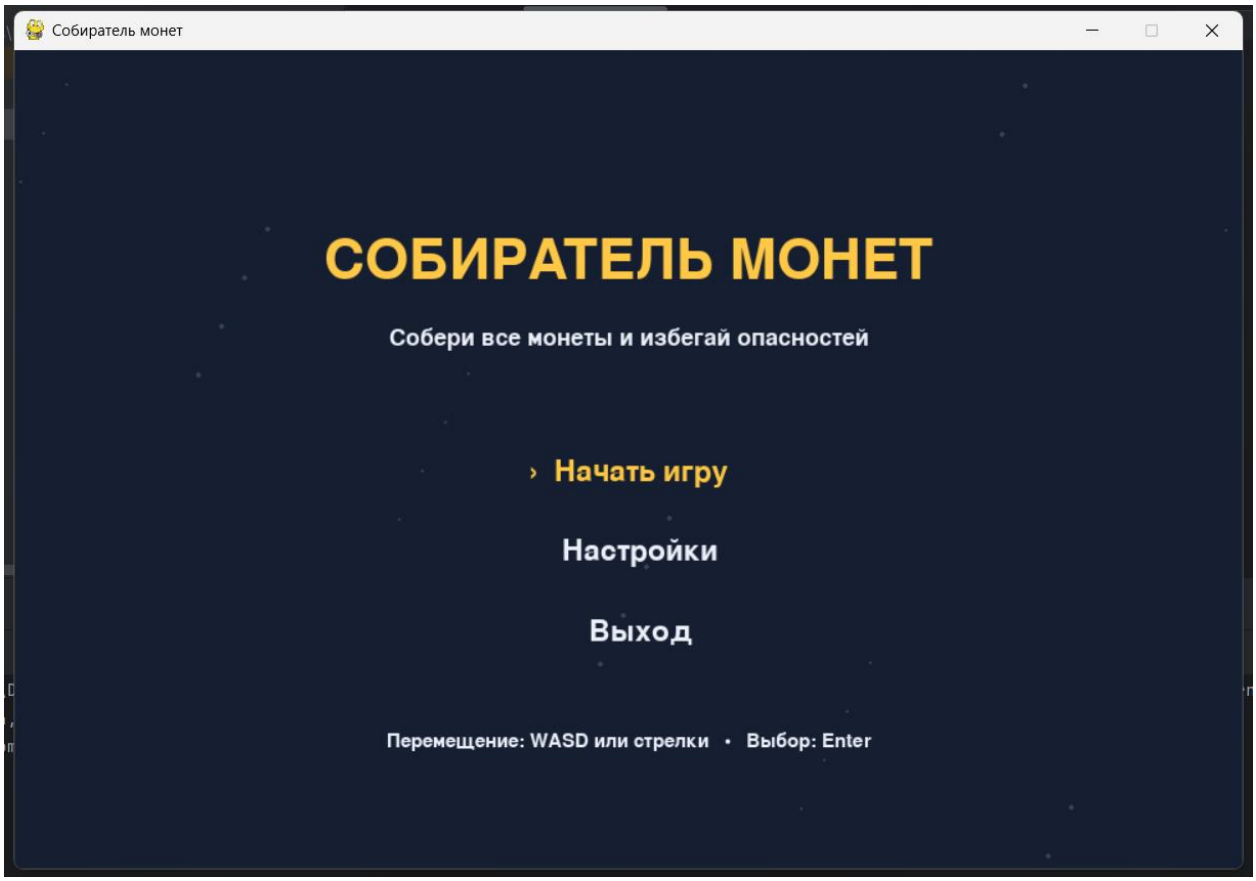


Рисунок 1. Начальный экран игры

2) Пройти и собрать все монеты.

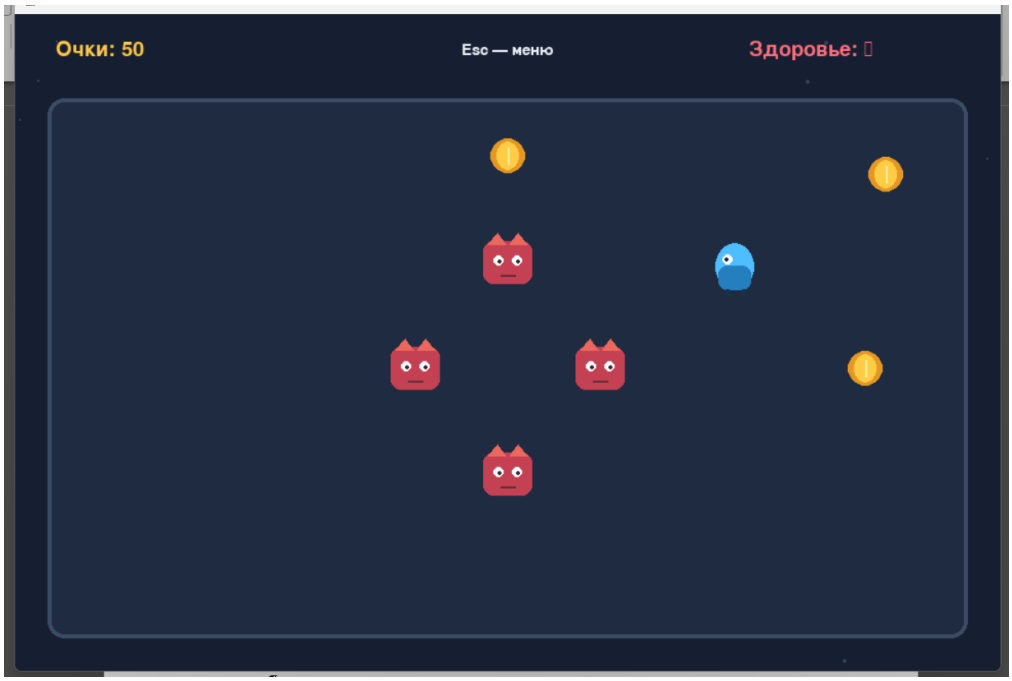


Рисунок 2. Процесс сбора монет

Ожидаемый результат: экран результата с сообщением "Все монеты собраны!" и корректным значением счёта.

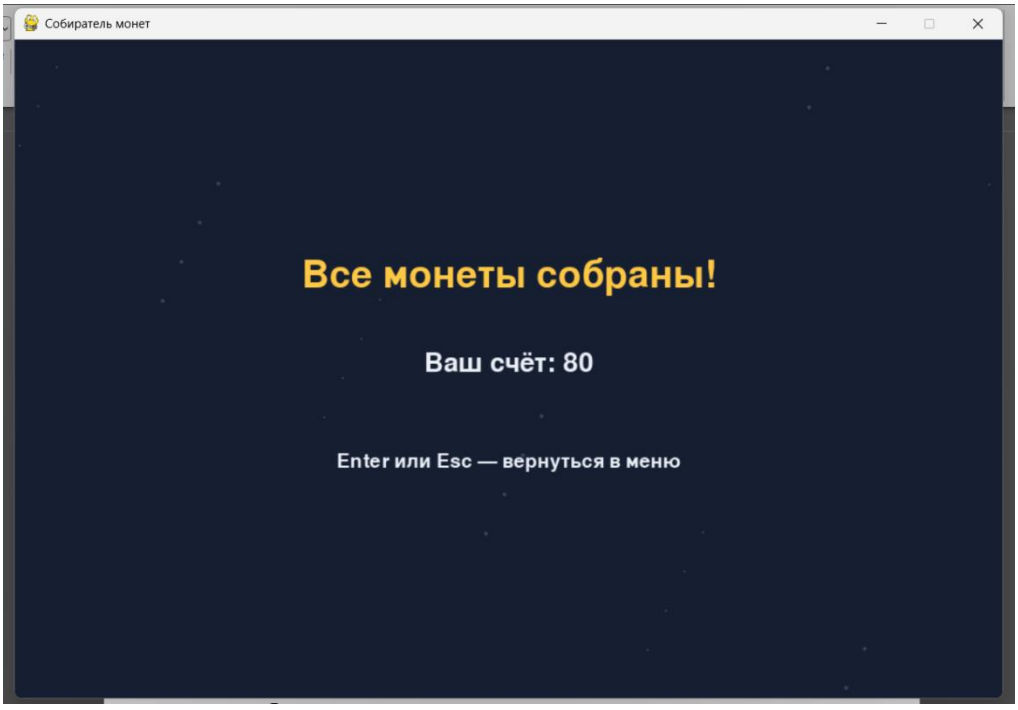


Рисунок 3. Конец игры

Результат соответствует ожидаемому поведению.

2. Тестирование «белого ящика»

Белый ящик: тестировщик использует знание внутренней структуры и кода для проверки логики, ветвлений, покрытий и важных внутренних состояний.

Имеем класс ArcadeObjectFactory, отвечающий за создание сущностей и размещение их на игровой карте:

```
class ArcadeObjectFactory(GameObjectFactory):
    def __init__(self) -> None:
        self._coin_prototype = Coin((0, 0))
        self._hazard_prototype = Hazard((0, 0))

    def create(self, kind: ObjectKind, position: tuple[int, int]) ->
GameObject:
        if kind is ObjectKind.PLAYER:
            return Player(position)
        if kind is ObjectKind.COIN:
            return self._coin_prototype.clone_at(position)
        if kind is ObjectKind.HAZARD:
            return self._hazard_prototype.clone_at(position)
        raise ValueError(f"Unsupported game object kind: {kind!r}")
```

Напишем юнит-тест, проверяющий, действительно ли возвращает фабрика заданные сущности:

```
class EntityTests(unittest.TestCase):
    def test_factory_creates_requested_entities(self):
        factory = ArcadeObjectFactory()
        self.assertIsInstance(factory.create(ObjectKind.PLAYER, (40,
40)), Player)
        self.assertIsInstance(factory.create(ObjectKind.COIN, (50, 50)),
Coin)
        self.assertIsInstance(factory.create(ObjectKind.HAZARD, (60,
60)), Hazard)
```

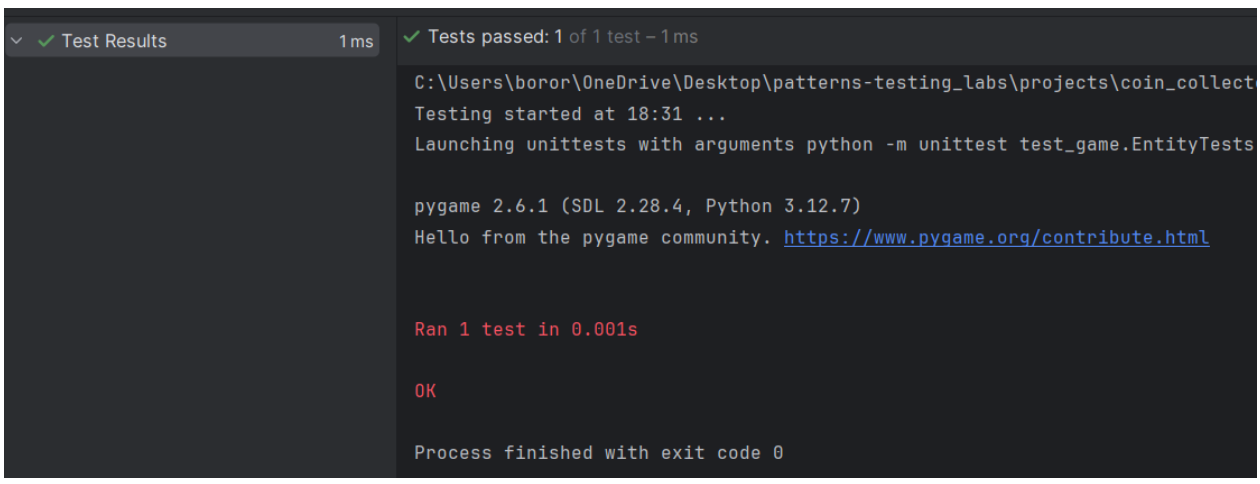


Рисунок 4. Результат тестирования фабрики объектов («белый ящик»)

Вывод:

Проделав работу, применил технологии тестирования «черного ящика» и «белого ящика» для разработанного приложения.